



Desafío Técnico - "Gestión de Tareas"



Objetivo

Crear una pequeña aplicación fullstack que permita gestionar tareas. El backend será construido con **Laravel**, el frontend con **Vue**, y todo debe correr dentro de **Docker**. El proyecto deberá conectarse a una base de datos **MySQL** y realizar un **CRUD completo** de tareas.



Requerimientos Técnicos

1. Backend (Laravel)

- Usar Laravel 10 o superior.
- Configurar el proyecto para correr con Docker (usando `sail` o `docker-compose`).
- Crear un modelo y migración para **Tarea** con los siguientes campos:
 - `id` (autoincremental)
 - `titulo` (string)
 - `descripcion` (text)
 - `estado` (enum: pendiente, en_progreso, completada)
 - `fecha_vencimiento` (nullable date)
 - `prioridad_id` (integer)
- Crear un modelo y migración para **Prioridad** (relacion 1 a muchos con Tarea) con los siguientes campos:
 - `id` (autoincremental)
 - `prioridad` (enum BAJA - MEDIA - ALTA)
- Crear un modelo y migración para **Etiquetas** (relacion muchos a muchos con Tarea) con los siguientes campos:
 - `id` (autoincremental)
 - `etiqueta` (enum DEV - QA - RRHH)
- Exponer un **API REST** con rutas para:
 - Listar todas las tareas
 - Crear una tarea
 - Editar una tarea
 - Eliminar una tarea
 - Ver una tarea específica
- Usar Resource Controllers.

2. Frontend (Vue.js)

- Crear un proyecto con Vue 3.
- Configurar un **store global** con Pinia o Vuex para manejar las tareas.
- Consumir el API de Laravel (puede usar axios o fetch).
- Crear una interfaz simple que permita:
 - Listar todas las tareas
 - Crear una nueva tarea (quiero poder asignarle una prioridad y una o varias etiquetas)
 - Editar una tarea existente
 - Eliminar tareas
 - Cambiar el estado de la tarea
- Bonus: Filtros por estado y fecha de vencimiento.

3. Base de Datos

- Conectarse a una base de datos MySQL en Docker.
- Crear las migraciones necesarias y poblar la base de datos con algunos registros de prueba (seeds opcional).

4. Docker

- Usar Docker para levantar todo el entorno:
 - PHP
 - MySQL
 - Nginx (opcional)
 - Laravel Sail (opcional)
 - Toda la app debe poder levantarse con un `docker-compose up`.
-



Entregables

- Repositorio en GitHub.
 - Instrucciones claras para levantar el entorno (`README.md` con pasos reproducibles).
 - Código claro y estructurado (se valoran buenas prácticas de nombres, organización, y componentes reutilizables).
-



Bonus

Siéntete libre de agregar lo que creas necesario para que tu app luzca **bonita**, sea **escalable** y **mantenible**. Algunas ideas:

- Diseño visual con Tailwind, Bootstrap o algún framework CSS.
- Manejo de errores global (tanto en frontend como backend).
- Validaciones adicionales en formularios.
- Arquitectura en capas (servicios, repositorios).
- Pruebas automatizadas.
- Autenticación básica (opcional).



Tiempo de entrega

Tienes **1 semana** para desarrollar el proyecto.

Cuando tengas listo tu repositorio en GitHub pásanos el link, **contáctanos para coordinar una demo** donde nos muestres cómo funciona tu aplicación, cómo la organizaste y puedas responder algunas preguntas sobre tu implementación. Ten a mano tu código y tu IDE de base de datos abierta.